ЛЕКСИЧЕСКИЕ КЛАССЫ ЯЗЫКА

№	Токен	Имя	Код токена	Формальный шаблон	Значение атрибута
1	package8	**pckg**	lcPckg	package8	0
2	{	**{**	lcStart	{	0
3	}	**}**	lcStop	}	0
4	var8	**var**	lcVar	var8	0
5	do8	**do**	lcDo	do8	0
6	if8	**if**	lcIf	if8	0
7	exit8	**exit**	lcExit	exit8	0
8	enddo8	**enddo**	lcEnddo	enddo8	0
9	typedef8	**typedef**	lcTypedef	typedef8	0
10	struct8	**struct**	lcStruct	struct8	0
11	точка с запятой	**;**	lcSemi	;	0
12	точка	**.**	lcDot	.	0
13	откр. скобка	**(**	lcOpPar	(	0
14	закр. скобка	**)**	lcClPar	)	0
15	присваивание	**ass**	lcAss	=	0
16	присваивание типа	**typeass**	lsTypeAss	<-	0
17	операция сравнения	**comp**	lcComp	< | <= | > | >= | == | !=	opL (<) opLe (<=) opG (>) opGe (>=) opE (==) opNe (!=)
18	логичИ	**and**	lcAnd	&	0
19	логичИЛИ	**or**	lcOr	|	0
20	логичНЕ	**not**	lcNot	!	0
21	логичИСТИНА	**true**	lcTrue	true	0

22	логичЛОЖЬ	**false**	lcFalse	false	0
23	аддитивная операция	**+**	lcAdd	+ | −	opAdd (+) opSub (−)
24	мультипл. операция	*	lcMult	* | /	opMult (*) opDiv (/)
25	идентификатор	**id**	lcId	l (l | d)* l − буква, d − цифра	указатель на запись в таблице символов
26	число	**num**	lcNum	d+ | d+.d+ d − цифра	указатель на запись в таблице символов
27	пробел	**sp**		(sp | tab | lf)+ sp − пробел, tab − табуляция, lf − перевод строки	токен не формируется
28	комментарий	**com**		<*X*> X − любой символ кроме *>	токен не формируется
29	маркер конца ввода	**eof**	lcEof		0
30	лексическая ошибка	**err**	lcErr		0

РБНФ

1. Программа = “**package8**” “IvanovIL” "**;**" РазделОбъявлений РазделОператоров.
2. РазделОбъявлений = [РазделТипов] РазделПеременных
3. РазделТипов = "**typedef8**" ОпределениеТипа "**;**" { ОпределениеТипа "**;**"}
4. ОпределениеТипа = Имя_типа “<-“ Тип
5. РазделПеременных = "**var8**" ОбъявлениеПеременной "**;**" { ОбъявлениеПеременной "**;**"}
6. ОбъявлениеПеременной = Тип Имя_переменной
7. Тип = Имя_Типа | СоставнойТип.
8. Имя = Буква { Буква | Цифра }.
9. СоставнойТип = "**struct8**" "{" Тип Имя_поля "**;**" { Тип Имя_поля "**;**"} "}"
10. РазделОператоров = "{" ПоследОператоров "}".
11. ПоследОператоров = Оператор "**;**" { Оператор "**;**"} .
12. Оператор = ОператорПрисваивания | УправляющийОператор.
13. ОператорПрисваивания = Переменная “=” Выражение.
14. Переменная = Имя_Переменной | ОбращениеКЭлементуСоставногоТипа.
15. ОбращениеКЭлементуСоставногоТипа = Переменная "." Имя_поля
16. УправляющийОператор ="**do8**" ПоследОператоров "**if8**" "**(**" Выражение "**)**" "**exit8**" "**enddo8**"
17. Выражение = ПростоеВыраж [ОперОтношения ПростоеВыраж].
18. ОперОтношения = "<" | "<=" | ">" | ">=" | “==”| “!=”.
19. ПростоеВыраж = ["**+**" | "−"] Терм { "**+**" | "−" Терм } | Терм { “|” Терм }.
20. Терм = Фактор { "*****" | "/" Фактор } | Фактор { “&” Фактор }.
21. Фактор = Целое ["**.**" Целое] | "**true**" | "**false**" | Переменная | "**(**" Выражение "**)**" | “!” Фактор.
22. Целое = Цифра { Цифра }.

ФОРМАЛЬНАЯ ГРАММАТИКА ЯЗЫКА

Prog → **pckg id ;** DeclPart StmtPart
DeclPart → TypePart VarPart | VarPart
TypePart → **typedef** LstTypeDef
LstTypeDef → TypeDef **;** | LstTypeDef TypeDef **;**
TypeDef → **id typeass** Type
VarPart → **var** LstVarDecl
LstVarDecl → VarDecl **;** | LstVarDecl VarDecl **;**
VarDecl → Type **id**
Type → **id** | StructType
StructType → **struct {** LstField **}**
LstField → Field **;** | LstField Field **;**
Field → Type **id**
StmtPart → **{** LstStmt **}**
LstStmt → Stmt **;** | LstStmt Stmt **;**
Stmt → AssignStmt | CtrlStmt
AssignStmt → Var **ass** Expr
Var → **id** | StructAccess
StructAccess → Var **. id**
CtrlStmt → **do** LstStmt **if (** Expr **) exit enddo**
Expr → SimpleExpr | SimpleExpr **comp** SimpleExpr
SimpleExpr → MathExpr | OrExpr
MathExpr → Term | **add** Term | MathExpr **add** Term
OrExpr → Term **or** Term | OrExpr **or** Term
Term → MultTerm | AndTerm
MultTerm → Factor | MultTerm **mult** Factor

AndTerm → Factor **and** Factor | AndTerm **and** Factor
Factor → **num** | **true** | **false** | Var | **(** Expr **)** | **not** Factor

ДЛЯ ГЕНЕРАТОРА

```
Prog = "pckg" "id" ";" DeclPart StmtPart “eof”.
DeclPart = TypePart VarPart | VarPart.
TypePart = "typedef" LstTypeDef.
LstTypeDef = TypeDef ";" | LstTypeDef TypeDef ";".
TypeDef = "id" "typeass" Type.
VarPart = "var" LstVarDecl.
LstVarDecl = VarDecl ";" | LstVarDecl VarDecl ";".
VarDecl = Type "id".
Type = "id" | StructType.
StructType = "struct" "{" LstField "}".
LstField = Field ";" | LstField Field ";".
Field = Type "id".
StmtPart = "{" LstStmt "}".
LstStmt = Stmt ";" | LstStmt Stmt ";".
Stmt = AssignStmt | CtrlStmt.
AssignStmt = Var "ass" Expr.
Var = "id" | StructAccess.
StructAccess = Var "." "id".
CtrlStmt = "do" LstStmt "if" "(" Expr ")" "exit" "enddo".
Expr = SimpleExpr | SimpleExpr "comp" SimpleExpr.
SimpleExpr = MathExpr | OrExpr.
MathExpr = Term | "add" Term | MathExpr "add" Term.
OrExpr = Term "or" Term | OrExpr "or" Term.
```

```
Term = MultTerm | AndTerm.
MultTerm = Factor | MultTerm "mult" Factor.
AndTerm = Factor "and" Factor | AndTerm "and" Factor.
Factor = "num" | "true" | "false" | Var | "(" Expr ")" | "not" Factor.!
```

ТАБЛИЦА СУО И СУТ

Продукция с действиями	Семантические правила	Семантические действия	Символ действия
Prog → **pckg id** {A1} **;** DeclPart StmtPart	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *AddType*(**id**.*pnt*, *catProgName*, *void*)	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *AddType*(**id**.*pnt*, *catProgName*, *void*)	*A*1
DeclPart → TypePart VarPart | VarPart			
TypePart → **typedef** LstTypeDef			
LstTypeDef → TypeDef **;** | LstTypeDef TypeDef **;**			
TypeDef → **id** {A2} **typeass** Type {A3}	**if id.** *pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *AddType*(**id**.*pnt*, *catTypeName*, Type.*type*)	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *Push*(**id**.*pnt*, *AttrSt*) // id.pnt	A2

		t1 := *Pop*(*AttrSt*) // Type.type
		t2 := *Pop*(*AttrSt*) // id.pnt
		AddType(*t2*, *catTypeName*, *t1*)
			A3

VarPart → **var** LstVarDecl			

LstVarDecl → VarDecl ; | LstVarDecl VarDecl ;			
VarDecl → Type **id** {A4}	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *AddType*(**id**. *pnt*, *catVarName*, Type.*type*)	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *t1* := *Pop*(*AttrSt*) // Type.type *AddType*(**id**.*pnt*, *catVarName*, *t1*)	A4
Type → **id** {A5}	**if id**.*pnt*.*Cat* ≠ *catTypeName* **then** *Type_Error*(2) Type.*type* := **id**.*pnt*.*Type*	**if id**.*pnt*.*Cat* ≠ *catTypeName* **then** *Type_Error*(2) *Push*(**id**.*pnt*.*Type*, *AttrSt*)	A5
Type → StructType			
StructType → **struct** { LstField **}** {A6}	StructType.*type* := *Record*(LstField.*list*)	*t1* := *Pop*(*AttrSt*) // LstField.list *Push*(*Record*(*t1*), *AttrSt*)	A6
LstField → Field ; {A7}	LstField.*list* := *MakeList*(Field.*fld*)	*t1* := *Pop*(*AttrSt*) // Field.fld *Push*(*MakeList*(*t1*), *AttrSt*)	A7

LstField → LstField
Field ; {A8}
	LstField.*list* := *Merge*(LstField1.*list*,
	Field.*fld*)
		t1 := *Pop*(*AttrSt*) // Field.fld
		t2 := *Pop*(*AttrSt*) // LstField1.list
		Push(*Merge*(*t2*, *t1*), *AttrSt*)
			A8

Field → Type **id** {A9}	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) Field.*fld* := *MakeField*(**id**.*pnt*, Type.*type*)	**if id**.*pnt*.*Cat* ≠ *catNoCat* **then** *Type_Error*(1) *t1* := *Pop*(*AttrSt*) // Type.type *Push*(*MakeField*(**id**.*pnt*, *t1*), *AttrSt*)	A9
StmtPart → **{** LstStmt **}**			

LstStmt → Stmt ; | LstStmt Stmt ;			
Stmt → AssignStmt | CtrlStmt			
AssignStmt → Var {A10} **ass** Expr {A11}	**if** Var.*type* ≠ Expr.*type* **then** *Type_Error*(4) *Gen*(*opAss*, Expr.*addr*, -1, Var.*addr*)		A10

		t1 := *Pop*(*AttrSt*) // Expr.addr
		t2 := *Pop*(*AttrSt*) // Expr.type
		t3 := *Pop*(*AttrSt*) // Var.addr
		t4 := *Pop*(*AttrSt*) // Var.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		Gen(*opAss*, *t1*, -1, *t3*)
			A11

Var → **id** {A12}	**if id**.*pnt*.*Cat* ≠ *catVarName* **then** *Type_Error*(3) Var.*type* := **id**.*pnt*.*Type* Var.*addr* := **id**.*pnt*	**if id**.*pnt*.*Cat* ≠ *catVarName* **then** *Type_Error*(3) *Push*(**id**.*pnt*.*Type*, *AttrSt*) *Push*(**id**.*pnt*, *AttrSt*)	A12
Var → StructAccess			
StructAccess → Var . **id**			
CtrlStmt → **do** LstStmt **if** (Expr) **exit enddo**			

Expr → SimpleExpr			
Expr → SimpleExpr **comp** SimpleExpr			
SimpleExpr → MathExpr			
SimpleExpr → OrExpr			
MathExpr → Term			
MathExpr → **add** Term			
MathExpr → MathExpr **add** Term			
OrExpr → Term **or** Term			
OrExpr → OrExpr **or** Term			
Term → MultTerm			
Term → AndTerm			
MultTerm → Factor			
MultTerm → MultTerm **mult** Factor			

AndTerm → Factor **and** Factor			
AndTerm → AndTerm **and** Factor			
Factor → **num**			
Factor → **true**			
Factor → **false**			
Factor → Var			
Factor → (Expr)			
Factor → **not** Factor			